



University of Information Technology and Sciences (UITS)

Department of Computer Science and Engineering

COURSE: Software Project Design and Development (CSE 416)

PROJECT PROPOSAL

Project Title: CampusCare

Centralized University Complaint Management System

Submitted To:

Al-Imtiaz

Associate Professor & Head
Ph.D. (Research Fellow), BUET
Department of CSE, UITS

**Submitted By (Group
Members):**

Rudro Antony Mrong

ID: 0432320005101059

Md. Masud Rahman

ID: 0432320005101064

Submission Date: August 2026

Contents

1	Executive Summary	2
2	Problem Statement	2
3	Project Objectives	2
3.1	Primary Objective	2
3.2	Quantifiable & Operational Objectives	2
4	Project Scope	3
4.1	In-Scope (Minimum Viable Product - MVP)	3
4.2	Out-of-Scope (Future Iterations - Version 2.0)	3
5	Feasibility Study	4
5.1	Technical Feasibility	4
5.2	Operational Feasibility	4
5.3	Economic Feasibility	4
6	Target Audience & Role Capability Matrix	5
7	Work Breakdown Structure (WBS) & Task Costing	5
8	Project Timeline & Gantt Chart	6
9	Budget & Cost Estimation	7
10	System Architecture & Technical Stack	7
11	Success Metrics & Evaluation Criteria	7

1 Executive Summary

CampusCare is a centralized 3-tier web application designed to track, manage, and resolve university facility maintenance issues, infrastructure reporting, and operational work orders. In traditional academic institutions, complaint reporting relies on unorganized channels such as phone calls, emails, and physical front-desk logbooks. This fragmentation leads to delayed issue resolution, lost requests, lack of status tracking for reporters, and poor operational visibility for university facility managers.

CampusCare introduces a single digital workflow governed by a multi-step state machine (**Submitted, Assigned, In Progress, Resolved, Closed, Reopened**). The platform provides role-based web interfaces for Students, Staff, and Administrators to ensure operational accountability, track Service Level Agreement (SLA) deadlines, and provide institutional reporting on campus infrastructure maintenance.

2 Problem Statement

Managing physical and technical infrastructure across campus lecture halls, dormitories, laboratories, and administrative offices involves thousands of active assets. Existing manual reporting workflows create several operational bottlenecks:

- **Unconsolidated Communication Channels:** Complaints submitted through emails or phone calls lead to misplaced requests and zero central record-keeping.
- **Lack of Reporter Status Tracking:** Students cannot track complaint progress after submission, resulting in redundant follow-up inquiries.
- **Unstructured Work Queues:** Maintenance staff lack prioritized digital queues, leading to uneven task distribution and missed service deadlines.
- **Limited Departmental Metrics:** Department heads lack quantitative data regarding active workloads, overdue tickets, resolution times, and recurring infrastructure faults.
- **Absence of Audit Records:** Without immutable database audit logs, tracing ticket history, verifying state changes, and evaluating historical staff performance remains difficult.

3 Project Objectives

3.1 Primary Objective

To design, develop, and deploy a responsive campus facility management platform that centralizes complaint submission, automated SLA deadline tracking, role-based dispatching, rate-limited submission APIs, and audit logging within a unified system.

3.2 Quantifiable & Operational Objectives

1. **Resolution Acceleration:** Reduce average complaint resolution lifecycle time across campus facility operations by 40%.

2. **Centralized Tracking:** Achieve 100
3. **SLA Compliance:** Maintain institutional SLA compliance above 85% across all complaint priority levels (Critical: 4h, High: 24h, Medium: 72h, Low: 168h).
4. **Database Query Performance:** Maintain database response latencies below 500ms for administrative reporting queries on datasets up to 10,000 records.
5. **Audit Traceability:** Maintain 100% audit coverage for ticket status transitions and staff assignment updates stored in an immutable audit ledger (`audit_logs`).

4 Project Scope

4.1 In-Scope (Minimum Viable Product - MVP)

The scope for the initial system release encompasses:

- **Authentication & Security:** Email/password authentication via `@supabase/ssr`, `HttpOnly` session cookie management, Next.js Edge proxy middleware for role-based route protection (`Student`, `Staff`, `Admin`), and Upstash Redis sliding-window rate limiting (`@upstash/ratelimit`) on submission endpoints.
- **Student Module:** Dashboard with metric cards, complaint submission form with custom animated dropdown pickers and photo proof uploads, human-readable Complaint ID generation (`CMP-YYYY-XXXX`), real-time ticket timeline tracking, and resolution confirmation/rejection workflow.
- **Staff Module:** Departmental ticket queues, visual SLA deadline warning indicators, state transition controls (`Assigned` → `In Progress` → `Resolved`), internal progress note logs, manual SLA target date overrides with reason notes, and completion photo proof uploads.
- **Admin Module:** Global master ticket table with multi-parameter filtering using `@tanstack/react-table`, ticket reassignment and priority override controls, user role promotion and department assignment desk, category/department taxonomy management, and operational analytics charts.
- **Database & Storage Infrastructure:** PostgreSQL relational schema with Row Level Security (RLS) policies utilizing `SECURITY DEFINER` helper functions to prevent policy recursion, foreign key constraints, automated ticket sequence triggers, and an immutable audit log table (`audit_logs`).

4.2 Out-of-Scope (Future Iterations - Version 2.0)

The following capabilities are excluded from the initial release and reserved for future project phases:

- Automated AI complaint classification and duplicate ticket detection.

- Real-time WebSocket push notifications and external transactional email dispatching (e.g., Resend / SendGrid API).
- Native mobile applications for iOS/Android operational staff (React Native / Expo).
- Multi-campus organizational hierarchy support across geographically separated university branches.

5 Feasibility Study

5.1 Technical Feasibility

The system leverages Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS, `@tanstack/react-table`, and Supabase (PostgreSQL). This stack provides standard SSR support, strong type safety, and clean separation of client and server logic. Supabase Row Level Security (RLS) combined with `SECURITY DEFINER` SQL helper functions enforces strict role-based data isolation at the database layer. Upstash Redis handles request rate limiting to protect public endpoints against abuse.

5.2 Operational Feasibility

The UI is built using the Caring Campus Green design system with responsive layouts supporting viewports down to 320px width and high-contrast text meeting WCAG standards. The system reflects standard university facility workflows (**Student reports** → **System/Admin assigns** → **Staff resolves** → **Student verifies**), requiring minimal training for campus maintenance personnel and students.

5.3 Economic Feasibility

Next.js, React, Tailwind CSS, and PostgreSQL incur zero software licensing fees. Hosting on Vercel and Supabase keeps operational overhead low, starting on free dev tiers and scaling to modest Pro tiers (\$45/month total) for production workloads, significantly reducing manual administrative labor costs.

6 Target Audience & Role Capability Matrix

Capability / Function	Student Role	Staff Role	Admin Role
Submit New Complaint	Yes	No	No
View Personal Submitted Complaints	Yes	No	Yes (All)
View Department Assigned Complaints	No	Yes	Yes (All)
Claim / Accept Ticket & Update Status	No	Yes	Yes
Override Ticket Priority & SLA Target	No	Yes	Yes
Append Progress Notes	Public Notes Only	Yes	Yes
Confirm or Reject Resolution		No	Yes (Override)
Assign or Reassign Staff/Department	No	No	Yes
Promote User & Assign Staff Department	No	No	Yes
View System Audit Logs	No	No	Yes (Read-Only)

Table 1: Role Capability Matrix across CampusCare User Roles.

7 Work Breakdown Structure (WBS) & Task Costing

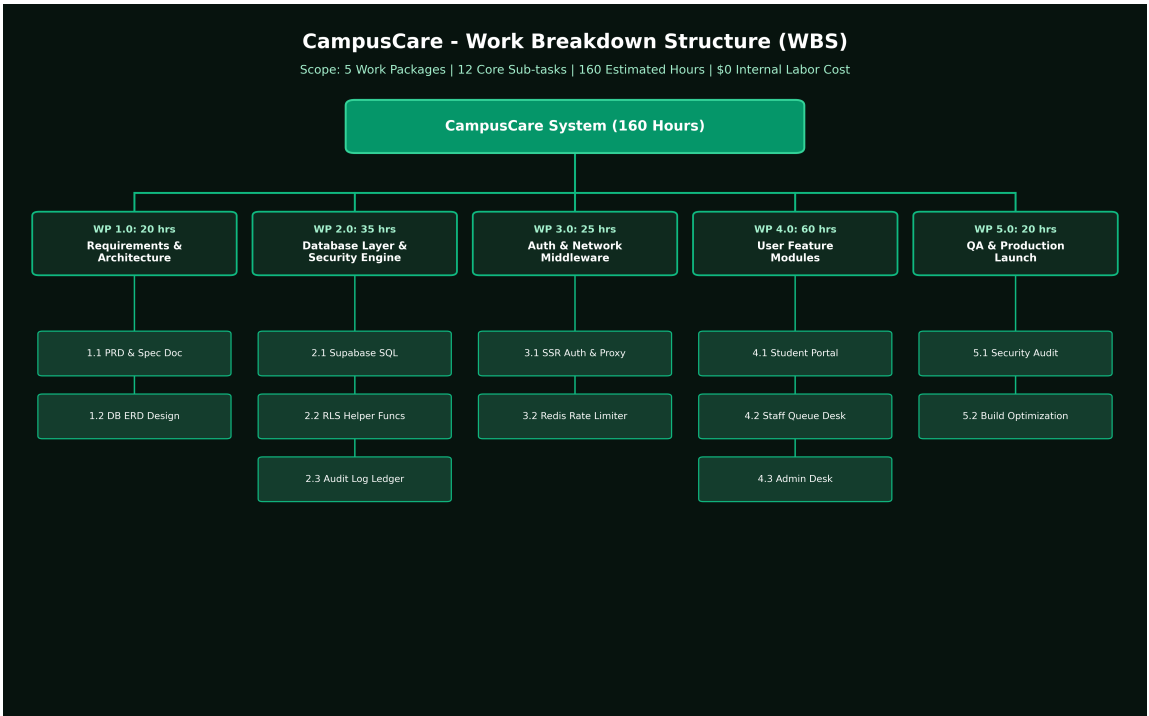


Figure 1: CampusCare Work Breakdown Structure (WBS) Tree Diagram.

Work Package	Title & Deliverables	Labor (Hours)	Cost (USD)
WP 1.0	Requirements & Architecture (PRD, Spec, ERD)	20 hrs	\$0.00
WP 2.0	Database & Security (PostgreSQL, RLS, Audit Log)	35 hrs	\$0.00
WP 3.0	Auth & Middleware (@supabase/ssr, Proxy, Redis)	25 hrs	\$0.00
WP 4.0	Core Modules (Student, Staff, Admin Desks)	60 hrs	\$0.00
WP 5.0	QA & Launch (Security Audit, WCAG, Build Optimization)	20 hrs	\$0.00
Total Project	5 Work Packages (12 Sub-tasks)	160 Hours	\$0.00

Table 2: Work Package Task & Labor Costing Breakdown.

8 Project Timeline & Gantt Chart

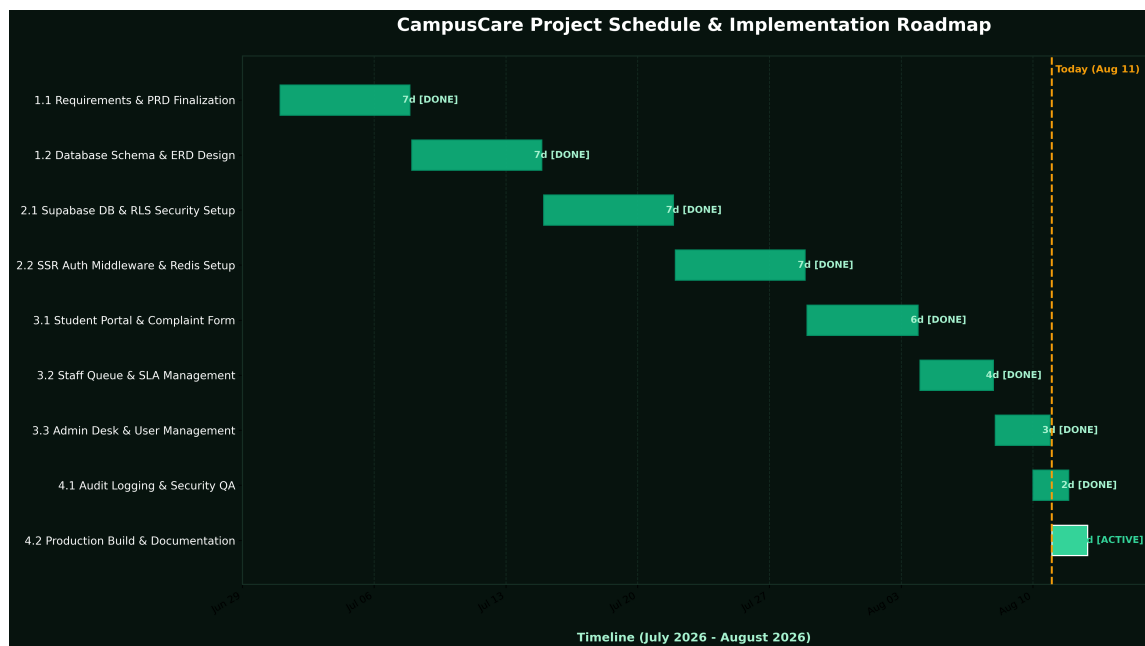


Figure 2: CampusCare Project Implementation Schedule & Roadmap.

9 Budget & Cost Estimation

Category	Component / Resource	Quantity	Annual Cost (USD)
Development	Internal Engineering Team	1 Team	\$0.00
Application Hosting	Vercel Hosting (Pro Tier)	12 Months	\$240.00
Database & Auth	Supabase Managed PostgreSQL (Pro Tier)	12 Months	\$300.00
Rate Limiting	Upstash Redis Cloud	12 Months	\$120.00
Domain & Network	Campus Subdomain (campuscare.university.edu)	1 Domain	\$0.00
SSL Certificate	Vercel / Let's Encrypt Automated SSL	1 Certificate	\$0.00
Buffer / Storage	Bandwidth & Storage Contingency Buffer	1	\$100.00
Total Annual Cost			\$760.00

Table 3: Annualized Cloud Infrastructure & Operational Budget Breakdown.

10 System Architecture & Technical Stack

- **Presentation Layer (Frontend):** Built using Next.js 16 (App Router), React 19, TypeScript, styled with Tailwind CSS using the Caring Campus Green design system. Data tables utilize `@tanstack/react-table`. Responsive layouts support mobile viewports down to 320px width.
- **Application Layer (Backend):** Server-side logic and API routes managed in Next.js. Server validation handled by Zod schemas. Session handling managed via `@supabase/ssr` with Edge proxy middleware. API protection handled by Upstash Redis sliding-window rate limiters.
- **Data & Storage Layer:** Relational database on Supabase PostgreSQL. Row Level Security (RLS) policies enforced at the database engine level. Uploaded attachments stored in Supabase Storage buckets with MIME-type validation.

11 Success Metrics & Evaluation Criteria

- **Resolution Speed:** Achieving at least a 90% completion rate for submitted complaints reaching `Closed` status within target SLA limits.
- **SLA Compliance:** Maintaining > 85% compliance with target resolution timeframes across all priority levels.
- **Query Performance:** Maintaining administrative master table query response times under 500ms for datasets up to 10,000 complaint records.
- **User Feedback:** Achieving an average user satisfaction rating of at least 4.0 out of 5.0 stars on closed ticket evaluations.